

Laboratorio CSV: Catálogo persistente

C++20 — catalog.csv

1. Modelo de dominio

Usa exactamente esta estructura:

```
struct Product {
    int id;
    std::string name;
    std::string category;
    double price;
    int stock;
    std::string note; // puede estar vacio
};
```

El archivo se llama `catalog.csv` y su cabecera exacta es:

```
id,name,category,price,stock,note
```

2. Archivo inicial

Incluye estas filas y agrega al menos dos productos propios:

```
1001,Monitor,Hardware,249.99,7,
1002,"Keyboard, TKL",Hardware,79.50,12,"Oferta, limitada"
1003,"Mouse ""Pro""",Hardware,39.90,20,"Dijo ""nuevo"" en caja"
1004,Cafe,Cocina,8.75,4,"Pequeno, 250 g"
```

La primera fila prueba un campo final vacío; las otras prueban comas y comillas dentro de campos.

3. Parser y serializer CSV

Tu código CSV debe ser general y soportar:

- Campos simples: `a,b,c`.
- Campos vacíos, incluso al inicio o al final: `a,,c`, `,b,`.
- Campos entrecomillados: `"a,b",c`.
- Comillas literales duplicadas: `"a""b",c` debe producir `a"b`.
- Detección de comillas sin cerrar.
- Cantidad arbitraria de campos; no hardcodees seis lecturas.

No se exige soportar saltos de línea reales dentro de un campo entrecomillado.

Implementa:

```
std::optional<std::vector<std::string>>
parseCSVLine(const std::string& line);

std::string escapeCSVField(const std::string& value);
```

```
std::string makeCSVLine(  
    const std::vector<std::string>& fields);
```

`parseCSVLine` devuelve campos ya desescapados o `std::nullopt`. `escapeCSVField` decide cuándo entrecomillar y duplica comillas literales. `makeCSVLine` construye la fila completa.

Para campos válidos debe cumplirse:

```
parseCSVLine(makeCSVLine(fields)) == fields
```

4. Conversión entre campos y Product

Una fila solo se acepta si:

- Tiene exactamente 6 campos.
- `id > 0`.
- El ID no está repetido entre los productos ya aceptados.
- `name` y `category` no están vacíos.
- `price >= 0`.
- `stock >= 0`.
- Cada conversión numérica consume el campo completo; "19abc" no vale como 19.

Implementa:

```
std::optional<Product>  
productFromFields(const std::vector<std::string>& fields);  
  
std::vector<std::string>  
productToFields(const Product& p);
```

5. Carga y guardado

Implementa una arquitectura equivalente a:

Función	Responsabilidad
<code>parseCSVLine</code>	línea CSV → campos
<code>escapeCSVField</code>	valor → campo CSV seguro
<code>makeCSVLine</code>	campos → línea CSV
<code>productFromFields</code>	campos → Product validado
<code>productToFields</code>	Product → campos
<code>loadCatalog</code>	archivo → vector<Product>
<code>saveCatalog</code>	vector<Product> → archivo reconstruido

Al cargar:

1. Abrir `catalog.csv`.

2. Leer y validar la cabecera exacta.
3. Procesar cada fila con el parser.
4. Si una fila falla, reportar número de línea y motivo, y continuar.
5. Guardar en el vector solo los `Product` válidos.

No uses `while (!eof())`.

Al guardar, reconstruye el archivo completo desde `std::vector<Product>`. No conserves las líneas originales para parchearlas después.

6. Operaciones en memoria

Implementa únicamente:

1. Listar productos.
2. Buscar un producto por ID.
3. Agregar producto, rechazando ID repetido.
4. Modificar `price`, `stock` o `note` de un producto existente.
5. Guardar y salir.

Toda modificación ocurre primero sobre `std::vector<Product>` en RAM.

7. Pruebas del codec

Verifica automáticamente estos casos:

Entrada	Resultado esperado
<code>a,b,c</code>	3 campos: a b c
<code>a,,c</code>	3 campos: a vacío c
<code>,b,</code>	3 campos: vacío b vacío
<code>"a,b",c</code>	2 campos: a,b c
<code>"a""b",c</code>	2 campos: a"b c
<code>"" ,c</code>	2 campos: vacío c
comilla sin cerrar	error

Además, verifica el round-trip para al menos cinco vectores difíciles con vacíos, comas y comillas.

8. Prueba final de persistencia

1. Ejecuta el programa y carga `catalog.csv`.
2. Agrega un producto cuyo `name` contenga una coma.
3. Haz que su `note` contenga al menos una comilla literal.
4. Modifica el stock de otro producto existente.

5. Guarda y termina el programa.
6. Abre `catalog.csv` y verifica el quoting.
7. Ejecuta el programa otra vez.
8. Comprueba que los objetos reconstruidos sean lógicamente iguales a los guardados.